

What's new in FastJet 3



Jets are self-aware

Jets from a cluster sequence know about properties deriving from the clustering itself:

```
// extract the constituents of a jet  
vector<PseudoJet> constituents = jets[0].constituents();
```

```
// get the scalar area of a jet  
double area = jets[0].area();
```



Composite jets

Jets can be joined

```
// subjects of a top candidate
PseudoJet W1[0];
PseudoJet W2[1];
PseudoJet b[2];

// build the top
PseudoJet W = join(W1,W2);
PseudoJet top = join(W,b);

// top constituents: all the initial particles in the jet
vector<PseudoJet> constituents = top.constituents();

// top pieces: the b and the W
vector<PseudoJet> pieces = top.pieces();
```



UserInfoBase

Define more complex user info

```
// user defined class
class MyInfo : public PseudoJet::UserInfoBase {
public:
    MyInfo(int id, int vertex) : pdg_id(id),
vertex_number(vertex){}
    int pdg_id, vertex_number; };

particle.set_user_info(new MyInfo(13,0)); // set the info

int id = particle.user_info<MyInfo>().pdg_id; // access it
```



Selectors

Cuts can easily be defined and applied

```
#include "fastjet/Selector.hh"
```

```
Selector pt_sel = SelectorPtMin(50.0);           // pt cut
Selector rap_sel = SelectorAbsRapMax(1.0);        // y cut
Selector sel_cuts = pt_sel && rap_sel;            // logical "and"
vector<PseudoJet> jets_within_cuts = sel_cuts(jets);
```

```
Selector nhardest_sel = SelectorNHardest(2);    // select 2 hardest
vector<PseudoJet> two_hardest_within_cuts =
    nhardest_sel(jets_within_cuts);
```

```
// alternatively, using a non-commuting "and"
Selector full_sel = SelectorNHardest(2) * sel_cuts;
vector<PseudoJet> two_hardest_within_cuts = full_sel(jets);
```



BackgroundEstimator

New classes to handle background estimation

```
#include "fastjet/tools/JetMedianBackgroundEstimator.hh"

Selector rap_sel = SelectorAbsRapMax(2.0);    // y range
JetMedianBackgroundEstimator bge(rap_sel);    // define bkgd. est.
ClusterSequenceArea csa(input_particles, jet_def, area_def);

bge.set_jets(csa.inclusive_jets()); // set jets to be used in bge

double rho    = bge.rho();                // extract value of rho
double sigma  = bge.sigma();               // extract value of sigma
```



Subtractor

And a new tool to handle background subtraction

```
#include "fastjet/tools/Subtractor.hh"

JetMedianBackgroundEstimator bge(...);
Subtractor subtractor(&bge);           // define subtractor

ClusterSequenceArea csa(input_particles, jet_def, area_def);
vector<PseudoJet> full_jets = csa.inclusive_jets();

vector<PseudoJet> subtr_jets = subtractor(full_jets);
```



Filter

```
#include "fastjet/tools/Filter.hh"
double Rfilt = 0.3;

// filtering
Selector sel_3hardest = SelectorNHardest(3);
Filter filter(Rfilt, sel_3hardest);
PseudoJet filtered_jet = filter(jet);

// trimming
Selector sel_aboveptfrac = SelectorPtFractionMin(0.05);
Filter trimmer(Rfilt, sel_aboveptfrac);
PseudoJet trimmed_jet = trimmer(jet);

// structure of result
vector<PseudoJet> kept_subjets = filtered_jet.pieces();
vector<PseudoJet> rejected_subjets =
    filtered_jet.structure_of<Filter>().rejected();
```



Taggers

Example of a boosted top tagger

```
#include "fastjet/tools/JHTopTagger.hh"

double delta_pt=0.1, delta_r=0.19;
JHTopTagger = top_tagger(delta_pt,delta_r);
top_tagger.set_top_selector(SelectorMassRange(150,200);
top_tagger.set_W_selector(SelectorMassRange(65,95);

PseudoJet tagged_jet = top_tagger(jet);    // top candidate

// extract structure
if (tagged_jet != 0) {
    PseudoJet W = tagged_jet.structure_of<JHTopTagger>().W();
    PseudoJet nonW = tagged_jet.structure_of<JHTopTagger>().non_W();
}
```